Sistem Güvenliği: Kesişimde Doğan Hatalar

ÖFY

October 2026

Özet

Kesişim noktalarında doğan hatalar, karmaşık sistemlerin olağan bir sonucudur. Parçalar sıkı bağlı olduğunda ve birbirleriyle beklenmedik biçimlerde etkileştiğinde, kazalar istisna olmaktan çıkar. Bu tür hatalara karşı en etkili savunma, hatayı test anında yakalamak değil, tasarım aşamasında imkânsız kılmaktır. Bunun yolları tek kural kaynağı, tiplerle ifade edilmiş model ve geçersiz durumun temsil edilemez hâle getirilmesidir. Yarış ve çökme hataları için en doğrudan araç deterministik simülasyon testidir; ayrıştırıcılar için ise fuzzing kullanılır. Alana Dayalı Tasarım, ortak dil ve kuralların modelde tutulması yoluyla bu hataların bir kısmını önler. Olay Güdümlü Mimari ise sıralama, tekrar ve yarıda kalma sorunlarını çoğaltır; bu yüzden ancak gerçek bir ihtiyaç doğduğunda seçilmelidir. Mimariden bağımsız olarak her projenin temeli aynı sırayla kurulur: önce yazılı kurallar, sonra bu kuralları zorlayan iskelet, en son bu ikisini sınayan testler.

Giriş

Hızlı büyüyen bir yazılım sisteminde her yeni özellik genellikle kendi başına test edilir. Eski özelliklerle buluştuğu noktalar ise test edilmez. Sonuç, tek tek doğru çalışan ama birlikteyken hata üreten parçalardır. Bu hatalar günlük kullanımda seyrek tetiklenir: iki işlem aynı saniyede gelir, deploy tam bir yükleme sırasında olur ya da iki kaynaktan aynı dosya adı gelir. Bu yüzden şikâyet gelmeden birikirler.

Bu soruna verilen ilk tepki genellikle daha çok test yazmak ya da daha gelişmiş bir mimariye geçmektir. Her iki tepki de eksiktir. Test, var olan hatayı bulur ama yenisinin doğmasını engellemez. Mimari değişikliği ise bazı hataları önlerken başka hataların ortaya çıkmasına yol açabilir.

Notta kullanılan merkezî terimler şöyle tanımlanır:

- **Kesişim**, iki ayrı özelliğin, kuralın ya da sürecin aynı veri veya aynı zaman aralığı üzerinde buluştuğu noktadır.
- **Değişmez** (*invariant*), hiçbir girdi ve koşulda bozulmaması gereken kuraldır.
- **Giriş kapısı**, sisteme verinin girdiği her noktadır. Örneğin zip yükleme, nesne depolama kovası, canlı yayın cihazı entegrasyonu ve web sayfası ayrı ayrı kapılardır.

Kapsam, web tabanlı ve asenkron medya işleme hattı içeren orta ölçekli sistemlerle sınırlıdır. Biçimsel doğrulama ve dağıtık veritabanı tasarımı kapsam dışıdır.

Hatanın Kesişimde Doğması

Normal Kazalar

Perrow, bazı sistemlerde kazanın kaçınılmaz olduğunu ileri sürmüştür (Perrow, 1984). Bu sistemlerin iki ortak özelliği vardır. Birincisi **sıkı bağlılıktır** (*tight coupling*): bir parçadaki değişiklik diğerine hemen ve tampon olmadan geçer. İkincisi **etkileşimsel karmaşıklıktır** (*interactive complexity*): parçalar tasarımcının öngörmediği yollarla birbirini etkiler. Perrow bu iki özelliği taşıyan sistemlerde yaşanan kazalara *normal kaza* adını vermiştir, çünkü bu kazalar tek bir parçanın arızasından değil, sağlam parçaların beklenmedik birleşiminden doğar.

Bir medya hattında iptal, render, yükleme ve yayın adımları birbirine sıkı bağlıdır. İptal komutu render bittikten sonra ama yükleme sürerken gelirse, her adım kendi kuralına göre doğru davranır. Yine de birleşik sonuç yanlış olabilir: video silinmiş ama yayın kaydı kalmış olur. Kesişim taramasının teorik temeli bu gözlemdir.

İsviçre Peyniri Modeli

Reason, kazaların birden fazla savunma katmanının aynı anda delinmesiyle oluştuğunu modellemiştir (Reason, 1997). Her katman (kod incelemesi, test, izleme, deploy denetimi) bir peynir dilimine benzetilir. Her dilimde delikler vardır ve kaza ancak bu deliklerin aynı hizaya gelmesiyle olur.

Modelin pratik sonucu, güvenliğin tek bir katmana yüklenmemesidir. Testin kaçırdığı hatayı çalışma anı değişmezi yakalayabilir. Onun da kaçırdığını izleme ve alarm yakalayabilir.

Sapmanın Normalleşmesi

Vaughan, Challenger uzay mekiği kazasını incelerken **sapmanın normalleşmesi** (*normalization of deviance*) kavramını tanımlamıştır (Vaughan, 1996). Kurallardan küçük sapmalar, felakete yol açmadıkları için zamanla kabul görür ve yeni normal hâline gelir.

Yazılımdaki karşılığı, aynı kuralın farklı giriş kapılarında ayrı ayrı yazılıp zamanla birbirinden ayrışmasıdır. Örneğin bir video uzantısı (`.MTS`) bir kapıda kabul edilir, diğerinde reddedilir. Her ayrışma tek başına küçüktür ve fark edilmez; birikince sistemin davranışı girişe göre değişir.

Safety-II

Hollnagel, güvenliği iki biçimde tanımlamıştır (Hollnagel, 2014). Safety-I, işlerin neden bozulduğunu sorar ve hatayı ortadan kaldırmaya çalışır. Safety-II ise işlerin normalde neden doğru gittiğini sorar. Bu bakışa göre sistemler çoğu zaman insanların fark etmeden yaptığı küçük düzeltmeler sayesinde ayakta durur.

Örneğin bir operatör, belirli bir kaynaktan gelen dosyaları yüklemeden önce elle yeniden adlandırıyor olabilir. Bu alışkanlık yazılı değildir ama bir hatayı

sürekli önler. Otomasyon bu adımı ortadan kaldırdığında hata ilk kez görünür hâle gelir.

Bulguların Ölçülmesi ve Değerlendirilmesi

Risk Tabanlı Test

Risk tabanlı test (*risk-based testing*), test çabasının risklere göre dağıtılmasıdır. Risk burada bir hatanın görülme olasılığı ile etkisinin birleşimi olarak tanımlanır (ISTQB, 2018). Hata sınıfının sistemde daha önce yaşanmış olmasını öncelik ölçütü yapan bir test düzeni, bu yaklaşımın bir uygulamasıdır. Yaşanmışlık, olasılığın ampirik bir göstergesidir.

Hata Kümelenmesi

Hatalar koda eşit dağılmaz, az sayıda modülde yoğunlaşır. Fenton ve Ohlsson, büyük bir telekom sisteminde hataların çoğunun modüllerin küçük bir bölümünde toplandığını göstermiştir (Fenton ve Ohlsson, 2000). Birden çok giriş kapısının aynı kuralı uyguladığı katmanlar ve yeniden işleme (*redo*) mantığı bu tür kümelerin tipik örnekleridir.

Pratik sonucu şudur: Bir taramada hata bulunan modül, bir sonraki taramada da öncelikli olarak incelenir.

Hata Keşif Eğrisi

Bir sistemde art arda yapılan taramalar giderek daha az hata bulur. Bu gözlem **yazılım güvenilirlik büyüme modelleri** (*software reliability growth models*) ile tanımlanmıştır. Goel ve Okumoto'nun modeli bu alanın klasiğidir (Goel ve Okumoto, 1979). Model, bulunan hata sayısının zamanla doyuma ulaşan bir eğri çizdiğini varsayar. Eğri düzleştiğinde kalan hata sayısının azaldığı sonucu çıkarılır. Eğri düzleşmiyorsa sistem hâlâ yeni hata üretiyor demektir.

Hizmet Düzeyi Hedefi ve Hata Bütçesi

Hizmet düzeyi hedefi (*Service Level Objective*, SLO), bir sistemin belirli bir ölçüde ne kadar iyi çalışması gerektiğini sayıyla tanımlar. Örneğin "paketlerin yüzde 99,5'i hatasız render edilir" bir hizmet düzeyi hedefidir. Hedefin dışında kalan pay ise **hata bütçesidir** (*error budget*). Bütçe dolduğunda yeni özellik geliştirme durdurulur ve güvenilirlik çalışmasına geçilir (Beyer ve diğ., 2016).

Bu yaklaşım, bir taramada bulunan hataların hangisinin "ağır" sayılacağı tartışmasını sayıya bağlar. Hedefi tehdit eden hata ağırdır, etmeyen hata uç durumdur.

Suçsuz Olay Sonrası Analiz

Suçsuz olay sonrası analiz (*blameless postmortem*), bir olaydan sonra hatayı yapan kişiyi değil, hatayı mümkün kılan koşulları arar (Beyer ve diğ., 2016). Suçlamanın olmaması, ekip üyelerinin olayın gerçek ayrıntılarını saklamamasını sağlar.

"5 Neden" (*5 Whys*) yöntemi bu analizlerde sık kullanılır. Yöntemde kök nedene ulaşmak için art arda "neden" diye sorulur. Ancak tek bir neden zinciri kurduğu için eleştirilir: kesişim hataları tek bir nedene değil, birden fazla koşulun aynı anda gerçekleşmesine dayanır.

Hatayı İmkânsız Kılmak

Poka-yoke

Poka-yoke, Toyota üretim sisteminde geliştirilmiş bir hata önleme ilkesidir (Shingo, 1986). İlke, hatanın sonradan denetimle yakalanması yerine, yapılmasının fiziksel olarak imkânsız kılınmasını önerir. Bir fişin prize yalnızca tek yönde takılabilmesi bunun gündelik örneğidir.

Geçersiz Durumu Temsil Edilemez Kılmak

Poka-yoke'nin yazılımdaki karşılığı, geçersiz durumların tip sistemiyle ifade edilemez hâle getirilmesidir. Minsky bu ilkeyi "geçersiz durumları temsil edilemez kıl" (*make illegal states unrepresentable*) biçiminde formüle etmiştir (Minsky, 2011).

Örneğin bir iş nesnesi hem `iptal_edildi` hem `yayında` alanlarını ayrı boolean değerler olarak taşıyorsa, ikisinin aynı anda doğru olması mümkündür. Tek bir durum alanı (`beklemede`, `render`, `yükleniyor`, `yayında`, `iptal`) kullanıldığında bu çelişki tip düzeyinde ortadan kalkar.

Doğrulama Değil, Ayrıştırma

King, "doğrulama değil, ayrıştırma" (*parse, don't validate*) ilkesini önermiştir (King, 2019). Doğrulama, ham veriyi kontrol eder ama ham hâlde bırakır. Bu yüzden aynı kontrol kodun farklı yerlerinde tekrar yapılmak zorunda kalır. Ayrıştırma ise ham veriyi kontrol ederken daha dar bir tipe dönüştürür. Örneğin bir dosya adı `string` olarak değil, yalnızca kabul edilen uzantıları taşıyabilen bir `KabulEdilenMedya` tipi olarak döner. Bu dönüşümden sonra kodun geri kalanı veriyi yeniden kontrol etmez.

Tek Kural Kaynağı

Birden çok giriş kapısının ayrışmasının kalıcı çözümü daha çok test değil, kuralın tek bir yerde yazılmasıdır. Dört kapı da aynı kabul modülünü çağırdığında kuralın ayrışması mümkün olmaz. Kapılar arasında kalan farklar ise artık yalnızca taşıma biçimiyle ilgilidir; zip açmak ya da kovadan indirmek gibi.

Bölme Duvarı ve Yük Atma

Bölme duvarı (*bulkhead*), adını gemilerdeki su geçirmez bölmelerden alır (Nygard, 2007). Sistem kaynakları (iş parçacıkları, bağlantılar, işçi süreçleri) bölümlere ayrılır. Böylece bir bölümün taşması diğerlerini etkilemez. Örneğin render işçileri ile API sunucusu aynı bağlantı havuzunu paylaşmaz.

Yük atma (*load shedding*), aşırı yük altında bazı isteklerin bilerek reddedilmesidir. **Geri basınç** (*backpressure*) ise tüketicinin üreticiyi yavaşlatabilmesidir.

İkisi de sistemin aşırı yük altında tamamen çökmesi yerine kontrollü biçimde yavaşlamasını sağlar.

Tamamlayıcı Test Türleri

Fuzzing

Fuzzing, bir programa çok sayıda rastgele ya da bozuk girdi verilerek çökme ve beklenmeyen davranışların aranmasıdır. Yöntemin ilk sistematik uygulaması Miller ve arkadaşlarının UNIX araçları üzerine çalışmasıdır (Miller, Fredriksen ve So, 1990). Günümüzde yaygın olan biçimi **kapsama güdümlü fuzzing**'dir (*coverage-guided fuzzing*). Bu biçimde araç, yeni kod yollarını açan girdileri saklar ve onları değiştirerek ilerler. AFL ve libFuzzer bu türün bilinen araçlarıdır.

Fuzzing en çok ayrıştırıcılarda verim verir: XML dosyaları, medya dosyası başlıkları ve zip arşivleri. Elle hazırlanmış bir bozuk medya korpusu sınırlı sayıda örnek içerir. Fuzzing ise bu korpusu otomatik olarak genişletir.

Deterministik Simülasyon Testi

Deterministik simülasyon testi (*deterministic simulation testing*), bütün sistemin tek iş parçacığında, sahte saat, sahte ağ ve sahte disk ile koşturulmasıdır. FoundationDB veritabanı bu yöntemle geliştirilmiştir (Zhou ve diğ., 2021).

Yöntemin gücü tekrar üretilebilirlikten gelir. Gerçek ortamda "iki yeniden işleme isteği aynı saniyede geldi" durumu rastlantısaldır ve tekrar üretilemez. Simülasyonda ise olayların sırası bir rastgele sayı tohumuna (*seed*) bağlıdır. Aynı tohum aynı sırayı üretir. Böylece binlerce farklı sıralama denenebilir ve hata bulunduğunda tam olarak tekrar oynatılabilir. Yarış ve çökme hataları için en doğrudan araç budur.

Kanarya Sürümü ve Gölge Trafik

Kanarya sürümü (*canary release*), yeni sürümün önce kullanıcıların küçük bir kesimine açılmasıdır. Hata oranı izlenir ve sorun yoksa sürüm kademeli olarak genişletilir (Beyer ve diğ., 2016).

Gölge trafik (*shadow traffic*) ise gerçek trafiğin bir kopyasının yeni sürüme de gönderilmesidir. Yeni sürümün yanıtları kullanıcıya ulaşmaz, yalnızca eski sürümünkilerle karşılaştırılır. Bu yöntem, üretim ortamında yapılan bir diferansiyel test sayılır.

Mimari Seçimi

Alana Dayalı Tasarım

Alana Dayalı Tasarım (*Domain-Driven Design*, DDD), yazılımın yapısını iş alanının kavramlarına göre kurmayı öneren bir yaklaşımdır (Evans, 2003).

Hataları Önleyen Kısmı

DDD'nin üç bileşeni kesişim hatalarının bir kısmını doğrudan önler:

- **Ortak dil** (*ubiquitous language*). Her kavramın tek ve net bir tanımı olur ve kodda da o adla geçer. Konuşma tanıma çıktısındaki bir *segment* ile dilbilgisel bir *cümle* farklı kavramlardır. İkisinin aynı sanılması, soundbite'ların yarım cümlede kesilmesine yol açar. Bu tür bir hata, özünde bir dil hatasıdır.
- **Kümeler ve değer nesneleri** (*aggregate*, *value object*). Kural, ilgili kavramın modelinin içinde tutulur. "Kabul edilen dosya türü" bir değer nesnesi olduğunda, her giriş kapısı aynı nesneyi kullanmak zorunda kalır.
- **Değişmezin modelde zorlanması**. "Tek kopya çalışır" gibi bir varsayım bir ayar satırında değil, modelin kendisinde korunur.

Sınırı

DDD'nin tamamı ağır bir yaklaşımdır. Sınırlı bağlam haritaları (*bounded context map*), depo katmanları (*repository*) ve katmanlı mimari, küçük ve orta ölçekli bir sistemde gereğinden fazla yapı üretebilir. Çoğu projede hafif hâli yeterlidir: ortak bir sözlük, tek bir kural modülü ve tiplerle ifade edilmiş değerler.

Olay Güdümlü Mimari

Olay Güdümlü Mimari (*Event-Driven Architecture*, EDA), bileşenlerin birbirini doğrudan çağırmak yerine olay yayımlayıp dinlediği bir yapıdır. Bileşenleri birbirinden ayırır, ancak kesişim sayısını artırır.

Çoğalttığı Hatalar

- **Sıra belirsizliği**. Olaylar gönderildikleri sırayla ulaşmayabilir. İptal olayının render bitiş olayından önce mi sonra mı geldiği sorusu her tüketicide ayrıca cevaplanmalıdır.
- **Tekrar teslim**. Çoğu mesaj altyapısı en az bir kez teslim (*at-least-once delivery*) garantisi verir. Aynı olay iki kez gelebilir. Tüketici idempotent değilse çift yayın ve çift ücret gibi hatalar olağan hâle gelir. Bu sorunun karşılığı idempotent alıcı (*idempotent receiver*) kalıbıdır (Hohpe ve Woolf, 2003).

- **Yarıda kalma**. Veritabanına kayıt yazıldıktan sonra ama olay gönderilmeden önce süreç çökebilir. Bu durumda kayıt vardır ama hiçbir tüketici bundan haberdar olmaz. Karşılığı, olayın kayıtla aynı işlem içinde bir tabloya yazıldığı **işlemsel giden kutusu** (*transactional outbox*) kalıbıdır (Richardson, 2018).
- **İzlenebilirlik kaybı**. Bir işin hangi bileşenlerden geçtiğini görmek zorlaşır. Her olayın bir ilişkilendirme kimliği (*correlation ID*) taşıması gerekir.

Seçilme Koşulu

EDA, idempotency, giden kutusu, sıralama denetimi ve ilişkilendirme kimliği ile birlikte kurulduğunda işe yarar. Tek başına kurulduğunda ise kesişim hatalarını artırır. Seçilmesi için şu ihtiyaçlardan en az biri gerçekten bulunmalıdır:

- Aynı olayı dinleyen çok sayıda bağımsız tüketici
- Bileşenlerin ayrı ayrı ölçeklenme gereği
- Bileşenlerin ayrı ekiplerce geliştirilmesi

Uzun süren ve asenkron medya işleri için durum makinesi taşıyan bir iş kuyruğu genellikle yeterlidir. Tam bir olay mimarisi gerektirmez.

Proje Temeli

Mimari seçiminden bağımsız olarak her projede kurulması gereken bir temel vardır. Bu temel, kurulma sırasına göre beş aşamada ele alınır.

Kod Öncesi Belge

Kod yazılmadan önce birkaç sayfalık bir belge hazırlanır. Belge şu bölümleri içerir:

- **Sözlük**. Her alan kavramının tek bir tanımı yazılır. Örneğin segment, cümle, paket ve yeniden işleme kavramları.
- **Durum makinesi**. Bir işin geçebileceği durumlar ve her olayın her durumda ne yaptığı bir tablo hâlinde yazılır. İptal, yeniden deneme ve çökme de bu tabloya dahildir. İptal ile yükleme gibi kesişim hatalarının çoğu, bu tablonun yazılmamış olmasından doğar.
- **Değişmezler**. Hiçbir koşulda bozulmaması gereken kurallar listelenir.
- **Varsayımlar**. "Tek kopya çalışır" ya da "dosya adları benzersizdir" gibi kabuller yazılır. Her birinin yanına kodda nasıl zorlandığı not edilir. Zorlanmayan bir varsayım, ileride ortaya çıkacak bir hatadır.
- **Dış sözleşmeler**. Dış servislerin ve cihazların ne döndürdüğü, gerçek örnek dosyalarla belgelenir.

Kod İskeleti

İlk özellik yazılmadan önce iskelet kurulur:

- Bütün giriş kapıları tek bir kabul modülünü çağırır.
- Alan kavramları tiplerle ifade edilir ve geçersiz durumlar temsil edilemez hâle getirilir.
- Her uzun iş bir idempotency anahtarı taşır. Aynı iş iki kez koşsa bile sonuç bozulmaz.
- İşler, süresi dolunca başka bir işçiye geçen bir kira (*lease*) ile kuyrukta tutulur. Böylece işçi ölürse iş kaybolmaz.
- Nazik kapanış (*graceful shutdown*) ve hazırlık sinyali (*readiness probe*) kurulur. Deploy, süren işi yarıda kesmez.
- Hiçbir hata yutulmaz. Her istek bir ilişkilendirme kimliği taşır ve loglar bu kimlikle aranabilir.
- Ayarlar bir şemaya göre doğrulanır. Eksik ya da yanlış ayar uygulamayı açılışta durdurur.

Ortam

- **Geliştirme ve üretim eşliği** (*dev/prod parity*). Geliştirme makinesi ile sunucu aynı imajla çalışır (Wiggins, 2011). Aksi hâlde, örneğin `ffmpeg`'in iki ortamda farklı sürümlerde olması, yalnızca sunucuda görünen hatalar üretir.
- Proje tek bir komutla, örnek veriyle birlikte ayağa kalkar.
- Testlerde elle yazılmış sahte veri yerine gerçekten gelmiş dosyalar kullanılır.

Test Düzeni

- Tip denetimi, lint ve kilit dosyası (*lockfile*) denetimi sürekli açıktır.
- Bir özelliğin bitmiş sayılması için kendi değişmezi ve en az bir kez kırmızı görülmüş testi bulunmalıdır.
- Ayrıştırıcılar özellik tabanlı test ve fuzzing ile sınanır.
- Giriş kapıları aynı girdiyle beslenip sonuçları karşılaştırılarak test edilir. Bu yönteme diferansiyel test denir.

Teslim ve Düzenli Bakım

- Deploy geri alma, migrasyon geri dönüşü ve yedekten dönüş en az bir kez gerçekten denenir.
- Riskli özellikler bir özellik bayrağı (*feature flag*) arkasında, önce küçük bir kesime açılır.
- Her büyük özellikten sonra ve müşteri sürümünden önce kesişim taraması yapılır.
- Her olaydan sonra suçsuz analiz yapılır ve bulunan hatanın varyantları kodun geri kalanında aranır.
- Kararsız (*flaky*) testler bir listede tutulur ve düzenli olarak temizlenir.

Ertelenecek Yapılar

Olay güdümlü mimari, mikroservisler, devre kesici, kaos testleri ve DDD'nin ağır bileşenleri baştan kurulmaz. Bu yapıların her biri belirli bir sorunu çözer ve kendi karmaşıklığını getirir. Sorun ortaya çıkmadan kurulduklarında yalnızca karmaşıklıklarını getirirler.

Sonuç

Kesişim hatalarına karşı savunmanın ağırlık merkezi test aşamasından tasarım aşamasına kaymalıdır. Test, var olan hatayı bulur. Tek kural kaynağı, tiplerle ifade edilmiş durum ve yazılı durum makinesi ise hatanın doğabileceği yüzeyi daraltır. Normal kazalar kuramı, bu daraltmanın neden gerekli olduğunu açıklar: etkileşim sayısı arttıkça sağlam parçalardan oluşan sistem bile hata üretir.

Mimari seçimin pratik karşılığı, ihtiyaca göre seçimdir. DDD'nin ortak dil ve modelde tutulan kural fikri hemen her projede yarar sağlar. Ağır bileşenleri ise ancak alan gerçekten karmaşık olduğunda gerekir. EDA'nın getirdiği ayrışma, sıralama ve tekrar sorunlarıyla ödenir. Bu bedelin ödenmesi ancak bağımsız tüketici, ayrı ölçekleme ya da ayrı ekip ihtiyacı gerçekten varsa anlamlıdır.

Önerilen temelin sınırlılığı, kod öncesi belgenin bakım gerektirmesidir. Sözlük ve durum makinesi kodla birlikte güncellenmezse kısa sürede yanıltıcı hâle gelir. Belgeyi kodla senkron tutmanın bir yolu, durum makinesini doğrudan kodda tanımlayıp belgeyi ondan üretmektir. Ancak bu çözüm her dil ve çerçevede kolay değildir.

Bir başka açık soru, deterministik simülasyon testinin maliyetidir. Yöntem, sistemin saat, ağ ve disk erişiminin baştan soyutlanmasını gerektirir. Var olan bir sisteme sonradan eklenmesi zordur. Bu yüzden yeni projelerde bu soyutlamanın baştan kurulup kurulmayacağı, proje temeline eklenmesi gereken bir karardır.

Kaynaklar

- Beyer, B., Jones, C., Petoff, J. ve Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fenton, N. E. ve Ohlsson, N. (2000). Quantitative Analysis of Faults and Failures in a Complex Software System. *IEEE Transactions on Software Engineering*, 26(8).
- Goel, A. L. ve Okumoto, K. (1979). Time-Dependent Error-Detection Rate Model for Software Reliability and Other Performance Measures. *IEEE Transactions on Reliability*, R-28(3).
- Hohpe, G. ve Woolf, B. (2003). *Enterprise Integration Patterns*. Addison-Wesley.
- Hollnagel, E. (2014). *Safety-I and Safety-II: The Past and Future of Safety Management*. Ashgate.
- ISTQB (2018). *Certified Tester Foundation Level Syllabus*. International Software Testing Qualifications Board.
- King, A. (2019). Parse, Don't Validate. *lexi-lambda.github.io*.

- Miller, B. P., Fredriksen, L. ve So, B. (1990). An Empirical Study of the Reliability of UNIX Utilities. *Communications of the ACM*, 33(12).
- Minsky, Y. (2011). Effective ML Revisited. *Jane Street Tech Blog*.
- Nygard, M. T. (2007). *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.
- Perrow, C. (1984). *Normal Accidents: Living with High-Risk Technologies*. Basic Books.
- Reason, J. (1997). *Managing the Risks of Organizational Accidents*. Ashgate.
- Richardson, C. (2018). *Microservices Patterns*. Manning.
- Shingo, S. (1986). *Zero Quality Control: Source Inspection and the Poka-yoke System*. Productivity Press.
- Vaughan, D. (1996). *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. University of Chicago Press.
- Wiggins, A. (2011). *The Twelve-Factor App*. 12factor.net.
- Zhou, J. ve diğ. (2021). FoundationDB: A Distributed Unbundled Transactional Key Value Store. *SIGMOD 2021*.